

Project Proposal (Proposed Solution) Report

Date:	08 October 2026	Team ID:	UNM1611JASON-SWTID-2026-4825
Project Name:	AI BlogNest API	Maximum Marks:	5 Marks

Project Overview:

Objective: To develop a robust, scalable, and secure RESTful backend API — **AI BlogNest API** — that harnesses Google Gemini 1.5 Flash Generative AI to automate blog article drafting and text summarization while providing complete author-protected CRUD data persistence in MongoDB.

Scope: The project encompasses user registration with bcrypt password encryption, JWT authentication, protected author routes, prompt engineering with customized text length and formatting constraints, automated category inferencing across 9 domains, and complete Thunder Client verification suites.

Problem Statement & Impact:

Description: Digital creators and developers face significant bottlenecks in producing technical content consistently. Existing writing workflows require 4 to 6 hours per article, and off-the-shelf generative AI web tools lack programmatic REST interfaces, database persistence, and author access controls.

Impact: Implementing AI BlogNest API eliminates writer's block, reduces content creation turnaround by over 90%, protects article ownership through tamper-proof JWT tokens, and provides developers with a turnkey microservice ready for web and mobile frontend integration.

Proposed Solution & Core Approach:

Approach: Built upon a clean Model-View-Controller (MVC) architectural pattern in Node.js and Express. The backend orchestrates client requests through dedicated controllers and middleware. When content generation is requested, the system constructs constrained prompts dispatched to Google Gemini 1.5 Flash, post-processes the output through a regex sanitization pipeline, auto-infers categories, and persists the result to MongoDB Atlas.

Key Features Implemented:

- **Gemini AI Blog Generator:** Generates high-quality 100-150 word articles complete with introduction, key insights, and conclusion.
- **AI Text Summarizer:** Condenses long articles into single-paragraph executive summaries.
- **Domain Classifier:** Auto-maps topics into Technology, Health, Business, Education, Finance, etc.
- **Cryptographic Security:** Bcrypt password salting (10 rounds) and 7-day HMAC-SHA256 JWT tokens.
- **Author-Protected CRUD:** Strict author ID checks ensuring only creators can modify or delete posts.

Proposed Solution (Continued) — Resource Requirements

Resource Requirements & Environment Specifications:

The table below details the hardware, software, and external cloud resources utilized to develop, test, and deploy AI BlogNest API.

Resource Type	Component / Tool	Specification / Allocation	Purpose in Solution
Hardware	Processor (CPU) RAM (Memory) Disk Storage	Multi-core x64 / ARM64 8 GB - 16 GB DDR4 256 GB NVMe SSD	Asynchronous Node.js execution, local MongoDB caching, and build toolchain execution.
Software Runtime	Node.js Runtime Express.js Framework npm Package Manager	v20+ LTS / v24 v4.18.2 v10+	Event-driven non-blocking I/O runtime and modular HTTP server framework.
Database	MongoDB Atlas Mongoose ODM	v7.0 Cloud Cluster v7.5.0 library	Persistent document storage with schema validation, indexing, and ObjectId relations.
Generative AI	Google Generative AI @google/generative-ai	gemini-1.5-flash / gemini-2.5-flash v0.24.1 SDK	State-of-the-art multimodal LLM providing sub-2s blog generation and text summarization.
Security & Tools	BcryptJS & JWT Thunder Client / Postman VS Code IDE	bcryptjs 2.4.3 / jsonwebtoken 9.0.2 Postman Collection v2.1 v1.90+ with ESLint	Cryptographic password hashing, bearer token security, API testing, and source code development.

Expected Project Outcomes:

- 1. Zero Friction API:** Fully operational REST API adhering to industry standards with 100% automated Thunder Client test pass rate.
- 2. High Performance AI:** Google Gemini API response time under 2 seconds with automatic formatting and category classification.
- 3. Enterprise-Grade Security:** Author-only write permissions, zero plaintext credential leakage, and centralized error logging.